La courbe de compromis Précision-Rappel

La courbe de compromis (ou tradeoff) entre la précision et le rappel est un outil essentiel pour évaluer les performances d'un modèle de classification binaire.

Principe

Cette courbe illustre la relation inverse qui existe entre la précision et le rappel lorsqu'on fait varier le seuil (threshold) de classification :

- **Précision** : proportion des prédictions positives qui sont correctes
- **Rappel** : proportion des cas positifs réels qui ont été correctement identifiés

Interprétation

- Un seuil élevé tend à :
    - Augmenter la précision (plus sélectif)
    - Diminuer le rappel (plus de faux négatifs)
- Un seuil bas tend à :
    - Diminuer la précision (plus de faux positifs)
    - Augmenter le rappel (plus permissif)

Utilité

Cette courbe permet de :

1. Visualiser le compromis entre précision et rappel
2. Choisir le seuil optimal selon les besoins métier
3. Comparer différents modèles de classification

Point optimal

Le "meilleur" seuil dépend du contexte :

- Pour un équilibre : chercher le point où précision ≈ rappel
- Pour minimiser les faux positifs : privilégier la précision
- Pour minimiser les faux négatifs : privilégier le rappel

La courbe aide à identifier visuellement ce point optimal selon les objectifs spécifiques du projet.

Code Python - Avoir la précision et le recal en fonction d'un threshold pour l'afficher dans un graphique

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import precision_recall_curve
```

```
import matplotlib.pyplot as plt
import pandas as pd

# 1. Création du modèle de régression logistique
modele = LogisticRegression()

# 2. Prédictions de probabilités avec validation croisée
#    → predict_proba retourne une probabilité pour chaque classe (0 et 1)
#    → .T transpose le résultat pour stocker séparément les colonnes dans
deux variables
data['proba_pas_cher'], data['proba_cher'] = cross_val_predict(
    modele,
    X,
    data['prix_encode'],
    cv=5,
    method='predict_proba'
).T  # Résultat transposé pour avoir proba pour la classe 0 et 1

# 3. Calcul des courbes de précision et rappel pour différents seuils
precision, rappel, seuils = precision_recall_curve(
    data['prix_encode'],
    data['proba_cher']  # Probabilité d'appartenir à la classe "cher"
)

# 4. Affichage d'un aperçu des premiers résultats
print(f'🔵 Précision (extrait): {precision[:5]}')
print(f'🟢 Rappel (extrait): {rappel[:5]}')
print(f'⚪ Seuils (extrait): {seuils[:5]}')

# 5. Création d'un tableau de synthèse
scores = pd.DataFrame({
    'seuil': seuils,
    'précision': precision[:-1],  # Même longueur que seuils
    'rappel': rappel[:-1]
})

# 6. Visualisation possible : plt.plot(scores['rappel'],
scores['précision']) pour générer la PR curve
scores.head()
```

Quand tu utilises cross_val_predict(..., method='predict_proba'), voici ce qu'il se passe concrètement :

🔁 À chaque itération de la cross-validation :

1. Le modèle est entraîné sur les K-1 folds (par exemple 80% des données si cv=5).
2. Le modèle prédit les probabilités sur le fold laissé de côté (les 20%).
3. Ces probabilités sont stockées pour les observations du fold de test uniquement.

Et ce processus est répété pour chaque fold, jusqu'à ce que chaque observation du dataset ait eu sa prédiction faite en dehors de son propre entraînement (🧼 donc pas de fuite de données).

De plus :

- **cross_val_predict** garantit que les probabilités sont restituées dans le même ordre que les index originaux.
- Donc, ligne par ligne, la proba dans data['proba_cher'][i] correspond bien à data['prix_encode'][i].

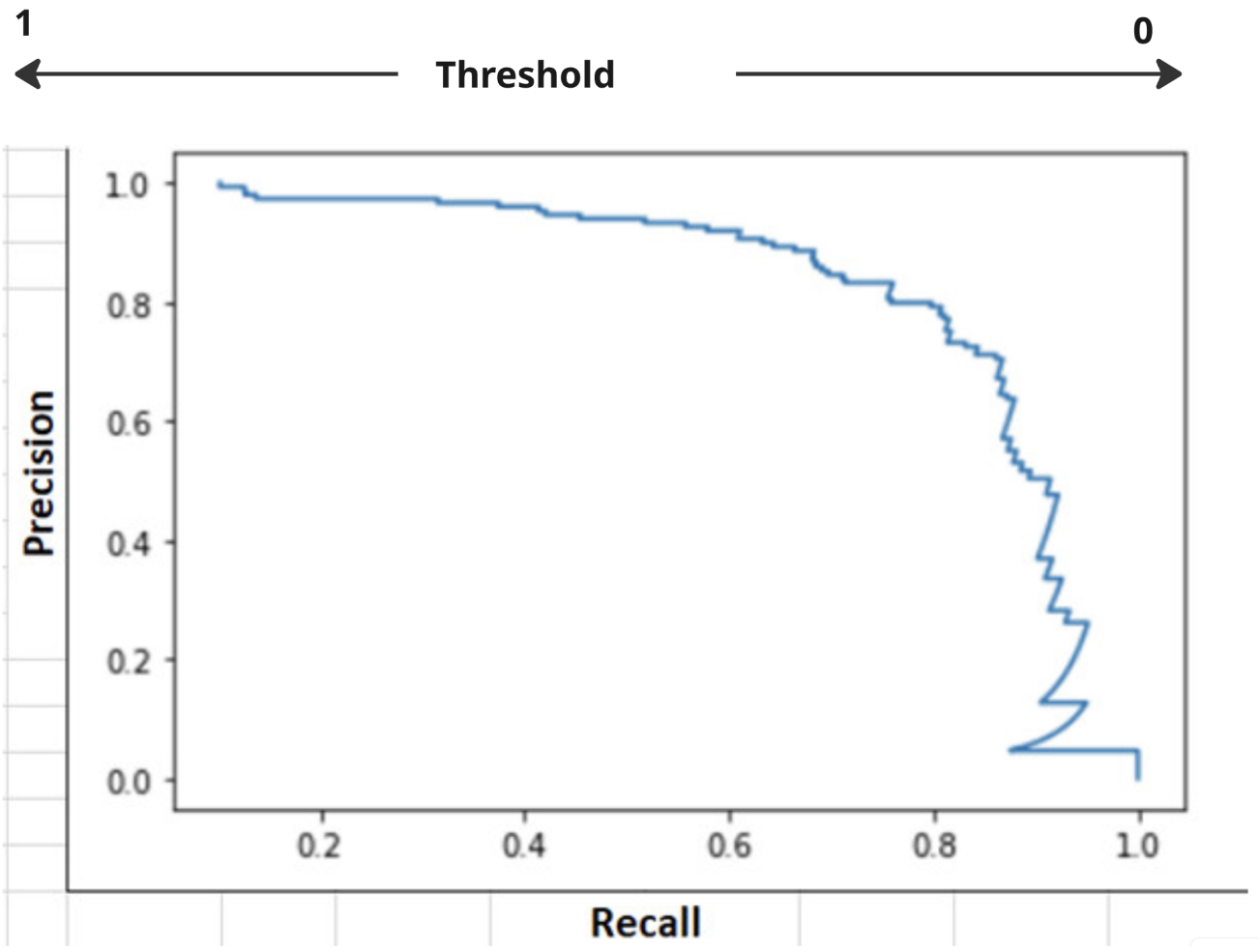
✅ Résultat final :

- Tu obtiens un vecteur complet de probabilités predict_proba de chaque observation, toutes faites comme si c'était sur des données "jamais vues".
- Si y contient deux classes (0 et 1), predict_proba retourne deux colonnes :
- proba_0 : proba d'être dans la classe 0 ~**proba_pas_cher**
- proba_1 : proba d'être dans la classe 1 (souvent ce qu'on garde) ~**proba_cher**

Afficher le graphique :

```
plt.plot(scores['rappel'],scores['precision'])
plt.ylabel('precision')
plt.xlabel('rappel')
```

On obtiendra le graphique suivant:



Trouvons le threshold qui nous permet d'avoir 80% de rappel

Nous avons besoin dans notre tableau des scores un rappel supérieur à 0.8 mais aussi dans le retour prendre le max des valeurs de threshold pour être sur d'avoir la précision la plus grande possible.

```
scores[scores['recall'] >= 0.8].threshold.max() = 0.25
```

Pour finir nous devons maintenant appliquer notre nouveau threshold:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import recall_score, precision_score, f1_score

# 1️⃣ Création et entraînement du modèle de régression logistique
modele = LogisticRegression()
modele.fit(X, data['price_range_encoded'])  # X = variables explicatives,
y = prix codé (0 ou 1)

# 2️⃣ Fonction personnalisée pour faire des prédictions avec un seuil
ajustable
def predictions_personnalisees(X, seuil_custom):
    # On récupère les probabilités prédites pour chaque observation
    probabilites = modele.predict_proba(X)  # Donne [P(classe 0), P(classe
1)] pour chaque observation

    # On ne garde que la probabilité que l'observation soit "cher" (classe
1)
    proba_cher = probabilites[:, 1]

    # On applique notre seuil : si la probabilité dépasse le seuil, on
classe comme 1, sinon 0
    return (proba_cher > seuil_custom).astype(int)

# 3️⃣ On fait des prédictions avec un seuil ajusté à 0.305539
predictions_modifiees = predictions_personnalisees(X, seuil_custom=0.0.25)

# 4️⃣ Évaluation des performances avec le nouveau seuil
print("Recall :", recall_score(data['price_range_encoded'],
predictions_modifiees))      # Sensibilité
print("Precision :", precision_score(data['price_range_encoded'],
predictions_modifiees))  # Précision
print("F1 Score :", f1_score(data['price_range_encoded'],
predictions_modifiees))          # F1
```

Nous avons besoin de faire cela car **predict** à par défaut un threshold de 0.5...